The Sandier–Serfaty triangular-lattice conjecture: an independent reproduction of a claimed finite certificate, an adversarial audit of the claimed reduction, and a Lean statement

(no proof of the conjecture is claimed)

Xavier Callens (SocrateAI Lab)

Preprint v1, 2026-10-25 (not peer reviewed)

doi:10.5281/zenodo.23653827 (not peer reviewed)

AI-assistance disclosure. This preprint, its Lean code, its Python and C++ harness code, the LeanFlow generator and the claim ledger were produced by AI agents (Claude, Anthropic) under the author's direction. The author is responsible for the content. No human has audited the Lean statements for adequacy, no human referee has checked the hand arguments, and every "independent" check below is made by an agent of the same AI pipeline as the producer, never by a person. The mathematics under study was itself produced by an AI system (the openai/math release) and is not peer reviewed. *Revision note.* This version incorporates the corrections required by an independent audit by a second agent instance of the same pipeline (misreported number in Section 4(b), fail-open LeanFlow checker for foreign sources in Section 4.4, conditional wording of Step 10, and non-blocking items); `AUDIT.md` and `REVISION.md` are in the paper directory.

Abstract

The Sandier–Serfaty conjecture asks whether the triangular lattice minimizes the renormalized Coulomb energy W of a point set in the plane against a uniform neutralizing background [1]. A preprint in the openai/math release [4] (AI-generated, not peer reviewed) claims a proof, by a reduction to square tori plus a finite certificate in exact interval arithmetic. We do *not* prove or verify that claim. We report three things. **(1) Independent reproduction of the finite certificate** [X]: the unchanged C++ certificate was re-executed on a different toolchain and host (`g++ (Ubuntu 11.4.0-1ubuntu1 22.04.3) 11.4.0`, Python 3.10.12); the wrapper exited 0, the terminal line was `OK`, source hashes were unchanged, and all 22 of 22 printed integer pairs equal the saved record. Three one-line loosenings of the certificate were run as negative controls: two are rejected by the program's own assertion and one passes (the outcomes of the last two were predicted in writing before they finished). The reproduction rests on the manuscript's analytic bounds and tests none of them. **(2) Adversarial audit of the reduction** [C]: by an AI agent, Sections 2–5 were checked step by step (31 numbered steps, 14 certificate-stage rows); no step was classified as a gap, with stated limits (the spline pieces outside $[p, R_0]$, the grid-stage values, and runtime trust are not independently checked). This is not a proof and not peer review. **(3) A Lean statement** [A for non-degeneracy lemmas only, C for fidelity]: the conjecture is stated as Lean `Prop`s (manuscript and Sandier–Serfaty normalizations, plus the explicit value); ten non-degeneracy lemmas are kernel-checked within the axioms {`propext`, `Classical.choice`, `Quot.sound`}, planted `sorry` and planted-axiom controls are caught; the statements themselves are *not proved*. We also add a Lean data-certificate generator to the LeanFlow project (88 tests pass), fail-closed for the sources it renders itself, which accepts the arithmetic of the positive run's printed pairs and rejects that of the loosened run. An independent audit found that the exported checker for *arbitrary* Lean sources was fail-open (a source using `#exit` and a forged axiom report was accepted); this is fixed in the checker (Section 4.4). Nothing here shows that the conjecture holds.

Tiers. [A]: kernel-checked Lean, axioms within the allowlist, planted controls caught; a Tier A tag attaches to the Lean statement, never to its gloss. [L]: a cited source whose statement we read on the cited page (proofs not re-checked). [C]: a human-style argument not machine-checked (here: by an agent). [X]: computation, including exact-integer and exact-rational computation whose toolchain is trusted; it never supports a mathematical claim by itself. Identifiers `SSA-A-0001` etc. refer to `ledger/ledger.json` (Section 6). Everything resting on the openai/math manuscript is conditional on it.

1 Setting, and what is and is not claimed

For $R > 1$ let $K_R = [-R, R]^2$ and fix cutoffs $\chi_R \in C_c^\infty(\mathbb{R}^2)$ with $0 \le \chi_R \le 1$, $\operatorname{supp}\chi_R \subset K_R$, $\chi_R = 1$ on K_{R-1} and $\sup_R \|\nabla\chi_R\|_\infty < \infty$. For a locally finite point set Λ with $\nu_\Lambda = \sum_{p\in\Lambda}\delta_p$ and bounded density $\sup_R \nu_\Lambda(B_R)/|B_R| < \infty$, the admissible class $\mathcal{A}_1$ consists of locally integrable fields E with

$$\operatorname{div} E = 2\pi(\nu_\Lambda - 1), \qquad \operatorname{curl} E = 0 \quad \text{in distributions.} \tag{1}$$

The renormalized energy is

$$W(E,\chi_R) = \lim_{\eta\downarrow 0}\Big[\tfrac{1}{2}\int_{\mathbb{R}^2\setminus\bigcup_p B(p,\eta)} \chi_R|E|^2 + \pi\log\eta\sum_{p\in\Lambda}\chi_R(p)\Big], \qquad W(E) = \limsup_{R\to\infty}\frac{W(E,\chi_R)}{4R^2}, \tag{2}$$

and $\Lambda_\triangle = \sqrt{2/\sqrt{3}}\,(\mathbb{Z}(1,0) + \mathbb{Z}(\frac{1}{2}, \frac{\sqrt{3}}{2}))$ has covolume one, with $E_\triangle = -\nabla h_\triangle$ the field of the mean-zero periodic solution of $-\Delta h_\triangle = 2\pi(\delta_0 - 1)$ on $\mathbb{R}^2/\Lambda_\triangle$. These are the definitions of the manuscript [4] (its eqs. for cutoffs, admissibility, local and whole energy), which follow Sandier–Serfaty [1] [L, SSA-L-0003]. The **conjecture** (SS2012 Conjecture 1, p. 5) is $W(E) \ge W(E_\triangle)$ for every $E \in \mathcal{A}_1$; SS2012 prove it among lattices (Theorem 2) and state that "its minimization in general remains open" [L, SSA-L-0001]. Betermin–Sandier [2] show that it is equivalent to the Brauchart–Hardin–Saff value of the next-order term of the optimal logarithmic energy of N points on S^2 [3] [L, SSA-L-0004, SSA-L-0005]. The manuscript [4] claims $W(E) \ge W(E_\triangle)$ and hence that sphere value.

What this paper does. It (i) re-executes the manuscript's finite certificate and attacks it with negative controls (Section 4); (ii) audits the reduction that feeds the certificate (Section 2, summarized in Section 5); (iii) states the conjecture in Lean (Section 3); (iv) improves LeanFlow, our Lean tooling, with a generator that turns printed interval data into Lean-checked arithmetic and fails closed on the sources it renders itself (Section 4.4; the audit-found hole in the checker for foreign sources and its fix are reported there). The rusty-SUNDIALS numerical library is not needed for this lane (the certificate is exact integer arithmetic, not an ODE problem).

What it does not do. It does not prove, verify or refute the conjecture, it is not peer review, and it did not search the literature for resolutions after the cited papers (the status "open" is that of the cited sources). A reproduction of an exact-integer computation shows that the program, as written, behaves as recorded on a second toolchain; whether it certifies the analytic inequalities it is meant to certify is an audit question (Section 5), not a computing one.

2 Step-by-step human demonstration of the claimed argument

This section is for a reader who knows the notation of Section 1. It restates, step by step, the argument of the manuscript *as re-derived by the audit agent*; it is not our proof, and each step carries (a) the audit's tier, (b) the audit rows (R/G/K/T) and ledger id, and (c) the matching Lean statement, which for most steps is "none". The only parts with Lean content are the definitions (Step 0), the value (Step 7) and the target statement (Step 10), plus the data certificates (Step 9). The logical shape is: Steps 1–4 reduce $W(E)$ on the plane to a lower bound $\mathcal{W}_n(h) \ge nC$ on square tori $T_n = \mathbb{R}^2/(\sqrt{n}\,\mathbb{Z}^2)$; Steps 5–9 are the manuscript's argument for that torus bound with $C = W(E_\triangle)$; Step 10 concludes.

Step 0: the objects [L, SSA-L-0003]. $\mathcal{A}_1$, $W(E,\chi)$, $W(E)$ and $\Lambda_\triangle$ as in (1)–(2).

Lean counterpart. `Admissible`, `AdmissibleBg`, `WeakDivE`, `WeakCurlFree`, `HasLocalEnergy`, `IsSquareCutoffFamily`, `WSq`, `triLattice` (Listing 1). Equations hold weakly against C_c^∞ test functions; the puncture limit is a relation (integrable on the punctured domain plus `Tendsto`), never `limUnder`. Non-degeneracy, [A, SSA-A-0001..0005]: the class $\mathcal{A}_1$ is non-empty ($E = -\pi x$, no charges), the background constant is load-bearing (controls $b = 0$, $b = -2\pi$ fail), both cutoff classes are non-empty, and the bases have covolume 1 and $2\pi/m$.

Step 1: local structure; the puncture limit exists [C, audit R1]. Near a charge p, $E_p^0 = \nabla \log|x-p| - \pi(x-p)$ has $\operatorname{div} E_p^0 = 2\pi\delta_p - 2\pi$ and $\operatorname{curl} E_p^0 = 0$; the difference $E - E_p^0$ has vanishing divergence and curl in the sense of distributions, so by Weyl's lemma it is smooth. Hence $E = (x-p)/|x-p|^2 + \text{smooth near } p$, $E \in L^q_{\mathrm{loc}}$ for $q < 2$, and $\frac{1}{2}|E|^2 - \frac{1}{2|x-p|^2}$ is integrable, which is exactly what makes the limit (2) exist and be finite.

Lean counterpart. Assumed, not proved: `HasLocalEnergy` states the limit as a hypothesis on the competitor. If the limit failed to exist for some admissible E, that E would drop out of the competitor class, so the Lean statement would be *weaker* (Section 3). Needs Weyl's lemma, absent from Mathlib at the pinned revision.

Step 2: normalization bridge [C+L, audit R2, R3; SSA-L-0003, SSA-L-0004]. Put $j = (-E_2, E_1)$. Then $\operatorname{div} j = -\operatorname{curl} E = 0$ and $\operatorname{curl} j = \operatorname{div} E = 2\pi(\nu - 1)$, so j is an SS2012 current with background $m = 2\pi$. The scaling (1.9) $j' = m^{-1/2} j(\cdot/\sqrt{m}) \in \mathcal{A}_1^{SS}$, $W(j) = m(W(j') - \frac{1}{4}\log m)$ [L, SS2012 p. 3] has the change of variables $y = \sqrt{m}\,x$ behind it: the puncture radius becomes $\eta' = \sqrt{m}\,\eta$, which produces $+\pi \log\sqrt{m}$ per charge, so $W(E, \chi_R) = W(j', \xi_t) - \frac{\pi}{2}\log m \sum_p \xi_t(p)$ with no area factor.

Lean counterpart. `SandierSerfatyStatement` (field E, square cutoffs, covolume one) and `SandierSerfatyOriginalStatement` (current j, ball cutoffs, Λ_m of covolume $2\pi/m$, every $m > 0$) are two separate `Prop`s. Their equivalence is [L/C] and is not formalized.

Step 3: $W(E) > -\infty$ on the plane [C+L, audit R4–R7; SSA-L-0001, SSA-L-0002]. For $E \in \mathcal{A}_1$ with $W(E) < \infty$ the local energies and charge counts are $O(t^2)$ on Q_t. SS2012 Lemma 4.7 (a Struwe-type dyadic L^r bound, $1 < r < 2$) then gives $\int_{Q_{t+A}} |j'|^r \le Ct^2(\log t)^{r/2}$, and Hölder with the coarea formula bounds the number of charges in a strip of width L around ∂Q_t by $C(t + t^{1+1/r}(\log t)^{1/2}) = o(t^2/\log t)$. SS2012 Proposition 4.9 (one measure $g \ge -C$ serving two cutoffs) turns the cutoff change $\xi_t \to \zeta_{Q_{t-L}}$ into an error $Cn(\log(2+n)+1) = o(t^2)$ with n the strip count. With SS Theorem 1(3) ($\min_{\mathcal{A}_1} W_U =: M$ finite) this gives $W(E) \ge 2\pi(M - \frac{1}{4}\log 2\pi) > -\infty$.

Lean counterpart. None. No Lean statement of Lemma 4.7 or Proposition 4.9 exists; both are cited [L] and their proofs are not re-checked.

Step 4: reduction to square tori [C+L, audit R8–R10; SSA-L-0001]. For a periodic field the energy per area equals the torus energy over the area (R8). On T_n, $E_{T_n}(E) = \mathcal{W}_n(h) + \frac{n}{2}|c|^2$ where $E = -\nabla h + c$ (R9): the harmonic part is constant and $\int \nabla h = 0$. SS2012 Theorem 1(4) supplies a minimizing sequence j_k for M, periodic with respect to axis-aligned squares of side $\sqrt{2\pi N_k}$; rescaling to $E_k(x) = \sqrt{m}(j_2, -j_1)(\sqrt{m}\,x)$ with $m = 2\pi$ gives fields on T_{N_k} with $E_T(E_k)/N_k = m(W_{\mathrm{ball}}(j_k) - \frac{1}{4}\log m) \to m(M - \frac{1}{4}\log m)$. Hence if $\mathcal{W}_n(h) \ge nC$ for every $n \ge 2$ and every configuration, then $W(E) \ge C$ for all $E \in \mathcal{A}_1$, and $\inf_{\mathcal{A}_1} W = W(E_\triangle)$ when $C = W(E_\triangle)$.

Lean counterpart. None (the torus theorem and this reduction are not stated in Lean).

Step 5: a torus minimizer, separation, Voronoi sectors [C, audit G1–G3]. (G1) $\mathcal{W}_n = \pi n R_n + \pi \sum_{i \ne j} G_n(v_i - v_j)$ (Green's identity; each site contributes $2\pi(-\log\eta + a_i)$), and lower semicontinuity on the compact T_n^n gives a minimizer with distinct points. (G2) A minimizer has torus separation $\ge r_* = 1/\sqrt{\pi}$. The tool is the disk profile $\varphi(x) = -\log(|x|/r_*) + \frac{\pi}{2}(|x|^2 - r_*^2)$ on the disk B of area one: $-\Delta\varphi = 2\pi(\delta - 1_B)$, $\varphi = \varphi' = 0$ at r_*, $\varphi > 0$ inside. A comparison function built from these profiles around the other sites satisfies $-\Delta\Psi \ge 2\pi/n > 0$ on the region where it differs from the potential Φ of the other sites; that region has measure $< n$, so each component has a boundary on which $\Psi = \Phi$, and the strong minimum principle (standard, not cited by the manuscript) contradicts minimality of Φ at the site if a neighbour were closer. We do not reproduce the details here (audit row G2). (G3) The Voronoi cells of the periodic lift have at most $6n$ face-edge incidences, each giving a sector $S(d, \alpha, \beta)$ with edge distance $d \ge r_*/2$, total area n and total angle $2\pi n$ (Euler's formula, $V - E + n = 0$, $3V \le 2E$). Separation is used only for the minimizer; the bound passes to all configurations by minimality.

Lean counterpart. None.

Step 6: dual identity and assembly [C, audit G4–G7]. For a trial potential H glued from radial data on Voronoi sectors (continuous off the sites, $H = -\log r + O(r^2)$ near them),

$$\mathcal{W}_T(h) = \sum_S J(S) + \tfrac{1}{2}\int |\nabla(h-H)|^2 \;\geq\; \sum_S J(S), \qquad J(S) = -2\pi\int_S H - \tfrac{1}{2}\int_S^{\mathrm{ren}} |\nabla H|^2$$

(G5; only the zero constant term of H enters the hole boundary term). A signed-endpoint lemma (G6) lets a sector whose foot-point lies outside its edge be treated: if $G(d,\cdot)$ is even, $2g_d \geq K$ on $x \geq 0$ and $\int \max(0,-G) \leq -K$, then the sum of two primitives is $\geq K$. Assembly (G7): $\sum_S J(S) \geq M_0 K + \lambda N + 2\pi\mu N \geq N(6K + \lambda + 2\pi\mu)$ since $K \leq 0$ and $M_0 \leq 6N$.

Lean counterpart. None.

Step 7: calibration on the triangular cell [C+X, audit K1–K8; SSA-X-0003, SSA-X-0004]. Let $a = \sqrt{2/\sqrt{3}}$, $p = a/2$, $R_0 = 2p/\sqrt{3}$, $x_0 = R_0/2$ (inradius, circumradius of the regular hexagon). The Green function has the expansion $h_\triangle(z) = -\log|z| + C_\triangle + \frac{\pi}{2}|z|^2 + \sum_{m\geq 1} c_m \operatorname{Re} z^{6m}$ on $0 < |z| < a$, $c_m = \frac{1}{6m}\sum_{z\in\Lambda_\triangle\setminus 0} z^{-6m}$ (K1; from the elliptic function $2\partial_z^2 h_\triangle = \wp$ and six-fold rotation). A sector integral reduces to $J = \int_{-\alpha}^{\beta}\big(f_0(d\sec\varphi) - \tan^2\varphi\, e(d\sec\varphi)\big)d\varphi$ (K4), and subtracting area A and angle θ gives the exact primitive form

$$J - \lambda A - \mu\theta = g_d(d\tan\alpha) + g_d(d\tan\beta), \qquad g_d(x) = \int_0^x G(d,s)\,ds, \quad G = \tfrac{d}{t^2}f - \tfrac{s^2}{dt^2}e,\; t = \sqrt{d^2+s^2} \qquad \text{(K5)}$$

with $f = f_0 - \lambda t^2/2 - \mu$. The constants λ, μ make (p, x_0) stationary (K6) and $K = 2g_p(x_0)$. On the regular hexagon the glued potential *is* $h_\triangle - C_\triangle$, so the square term vanishes and

$$W(E_\triangle) = 6K + \lambda + 2\pi\mu \qquad \text{(K7)}. \tag{3}$$

What remains (K8) are three scalar bounds, for $b = 28209/100000 < d_* = 1/(2\sqrt{\pi})$: $K < 0$; $2g_d(x) \geq K$ for $d \geq b$, $x \geq 0$; $\int_0^\infty \max\{0, -G(d,s)\}ds \leq -K$ for $d \geq b$. The exact value is Betermin–Sandier's closed form $W(E_\triangle) = \pi\log\pi - \frac{\pi}{2}\log 3 - 3\pi\log\Gamma(\frac{1}{3}) + \frac{3}{2}\pi\log 2 = \pi\log\frac{2\sqrt{2}\,\pi}{\sqrt{3}\,\Gamma(1/3)^3} \approx -4.1504128$ [L, SSA-L-0004].

Lean counterpart. `wTriClosedForm` $= \pi\log\big(2\sqrt{2}\,\pi/(\sqrt{3}\,\Gamma(1/3)^3)\big)$ and `SandierSerfatyValueStatement`. We evaluated `wTriClosedForm` with mpmath (30 digits) to -4.15041280767896337944, equal to the expanded closed form above; a transcription with $\sqrt{2\pi}$ in place of $\sqrt{2}\,\pi$ gives -5.94855 (negative control, [X]). The identification of this number with $W(E_\triangle)$ is [L+X] and is not formalized.

Step 8: infinitely many modes are cut at a finite number [C+X-exact, audit T1–T6; SSA-X-0005]. The coefficients satisfy $|c_m| \leq 7/(d\,a^d)$ for $d = 6m \geq 12$ (lattice-point count: $8v$ points per shell, $|w|^2 \geq \frac{3}{4}\max(|n|,|q|)^2$); every omitted mode and its first three derivatives are bounded by an explicit weight w_k ($k \geq 234$) with ratio test $\rho_i(234) < 1/10$, giving a weighted tail $\mathfrak{a} < 2\cdot 10^{-21}$, hence errors below 10^{-11} on f_0, e and their first two derivatives on $[b,l]$ ($2.4\cdot 10^{-13}$ actually used). Beyond $l = R_0 + 19/250$ the radial data are constant, so $e = 0$ and $f_0(t) = f_0(l) - \pi B(l)(t^2 - l^2)$ exactly. Every hand constant of the section was recomputed in exact rational arithmetic and passed, with four deliberately tightened negative controls failing.

Lean counterpart. None.

Step 9: the finite certificate [X exact-integer; arithmetic on printed integers [A]; SSA-X-0001, SSA-A-0007]. The program evaluates f_0, e, their derivatives and G with outward-rounded 144-bit interval arithmetic (Boost `cpp_int`) and asserts, in order: $-6.731 < \lambda < -6.729$, $K < 0$, $f(\ell) > 0$, slope $-\pi B(\ell) - \lambda/2 > 0$ (stages `calcLam`, `calcK`); derivative hulls $L_1, \dots, L_4 \geq \|G_d\|, \|G_s\|, \|G_{dd}\|, \|G_{ss}\|$ on $\mathcal{D} = [b, 0.697] \times [0, 0.65]$ (`bdsup`); on a 2186×3612 mesh, the outer assertion $m_* - E_0 - E_I > 0$ and the negative-part assertion $N_* + K + 0.65(S_h L_2/4 + D_h L_1/2) < 0$ (`global_grid`); and on a 601×4079 mesh, positivity of g_{dd}, g_{xx} and $\det \mathrm{Hess}\, g$ on the inner box $\mathcal{D}_0$ containing (p, x_0) (`inside_hess`), which with exact stationarity and convexity gives the inner minimum. Stage details and the stage↔inequality matching are in the audit (Section 5).

Lean counterpart. LeanFlow data certificates (Section 4.4): the 13 inequalities about the printed pairs of the positive run are accepted by core Lean `decide` [A, SSA-A-0007], scope: arithmetic on the printed outward-rounded integers only ([C, SSA-C-0005]).

Step 10: conclusion [L+C; conditional]. *If* the manuscript's analytic bounds, the cited results (whose proofs we did not re-check) and the certificate run are all correct, Steps 1–9 *would* give $\mathcal{W}_n(h) \geq nW(E_\triangle)$ (the torus theorem) and $W(E) \geq W(E_\triangle)$ for every $E \in \mathcal{A}_1$, which is what the manuscript states as its Theorem 1.1; we have established neither statement. Under that same hypothesis, Betermin–Sandier's Theorem 1.5 and its closed value -4.1504128 *would then identify* the next-order sphere constant with $C_{\mathrm{BHS}} = -0.0556053\ldots$, the $d = 2$ case of BHS Conjecture 4 [L, SSA-L-0004, SSA-L-0005]. The only parts of this implication that we checked are the cited statements (read on the cited pages) and the numerical identity $C = C_{\mathrm{BHS}}$ (to $4.0091e-51$, Section 4).

Lean counterpart. `SandierSerfatyStatement`, `SandierSerfatyValueStatement`: stated as `Props`, **not proved**. The sphere corollary is not stated in Lean.

3 The Lean statement

Pinned stack. Lean 4.34.1, Mathlib `d13f23b723b8a846827a245b89c10fc7d3f11612`, compiled with `scripts/certified_numerics/lean_pinned.py`. File `formal_cert/Programmes/SandierSerfatyStatement.lean` (sha256 `35add509990d50ebbdbbb70476fc652867d236a62534eda3d1021a414c21ae4d`); planted controls in `SandierSerfatyStatementControls.lean` (sha256 `aa94aca898a83529624931dd73dc7f0a3ab0ab31e78c9393de8c597661613895`). Listing 1 shows the definitions and the three statements verbatim, docstrings elided (the file has the docstrings and a longer header).

Listing 1: Definitions and the three statements (verbatim; docstrings elided). `SandierSerfatyStatement` is the manuscript normalization, `SandierSerfatyValueStatement` adds the explicit value, `SandierSerfatyOriginalStatement` is SS2012 Conjecture 1 for every $m > 0$. None is proved.

```
namespace ProgSandierSerfaty

open MeasureTheory Filter Topology Set
open scoped ContDiff

abbrev Plane := EuclideanSpace ℝ (Fin 2)

noncomputable def pd (i : Fin 2) (φ : Plane → ℝ) (x : Plane) : ℝ :=
  fderiv ℝ φ x (EuclideanSpace.single i 1)

def IsTestFn (φ : Plane → ℝ) : Prop := ContDiff ℝ ∞ φ ∧ HasCompactSupport φ

def LocFinite (Λ : Set Plane) : Prop := ∀ K : Set Plane, IsCompact K → (Λ ∩ K).Finite

def DensityBounded (Λ : Set Plane) : Prop :=
  ∃ C : ℝ, ∀ R : ℝ, 1 < R → ((Λ ∩ Metric.ball 0 R).ncard : ℝ) ≤ C * (Real.pi * R ^ 2)

def WeakDivE (b : ℝ) (Λ : Set Plane) (E : Plane → Plane) : Prop :=
  ∀ φ : Plane → ℝ, IsTestFn φ →
    -(∫ x, (E x 0 * pd 0 φ x + E x 1 * pd 1 φ x)) = 2 * Real.pi * (∑ᶠ p ∈ Λ, φ p) - b * ∫ x, φ x

def WeakCurlFree (E : Plane → Plane) : Prop :=
  ∀ φ : Plane → ℝ, IsTestFn φ → ∫ x, (E x 0 * pd 1 φ x - E x 1 * pd 0 φ x) = 0

def AdmissibleBg (b : ℝ) (Λ : Set Plane) (E : Plane → Plane) : Prop :=
  LocFinite Λ ∧ DensityBounded Λ ∧ LocallyIntegrable E volume ∧ WeakDivE b Λ E ∧ WeakCurlFree E

def Admissible (Λ : Set Plane) (E : Plane → Plane) : Prop := AdmissibleBg (2 * Real.pi) Λ E

def punctured (Λ : Set Plane) (η : ℝ) : Set Plane := (⋃ p ∈ Λ, Metric.ball p η)ᶜ

noncomputable def localEnergyAt (Λ : Set Plane) (E : Plane → Plane) (χ : Plane → ℝ) (η : ℝ) : ℝ :=
  (1 / 2) * (∫ x in punctured Λ η, χ x * ‖E x‖ ^ 2) + Real.pi * Real.log η * ∑ᶠ p ∈ Λ, χ p

def HasLocalEnergy (Λ : Set Plane) (E : Plane → Plane) (χ : Plane → ℝ) (w : ℝ) : Prop :=
  (∀ᶠ η in 𝒩[>] (0 : ℝ), IntegrableOn (fun x => χ x * ‖E x‖ ^ 2) (punctured Λ η)) ∧
    Tendsto (localEnergyAt Λ E χ) (𝒩[>] 0) (𝒩 w)

def sq (R : ℝ) : Set Plane := {x | |x 0| ≤ R ∧ |x 1| ≤ R}

def IsSquareCutoffFamily (χ : ℝ → Plane → ℝ) : Prop :=
  (∀ R : ℝ, 1 < R →
      ContDiff ℝ ∞ (χ R) ∧ HasCompactSupport (χ R) ∧ (∀ x, 0 ≤ χ R x ∧ χ R x ≤ 1) ∧
        tsupport (χ R) ⊆ sq R ∧ ∀ x ∈ sq (R - 1), χ R x = 1) ∧
    ∃ C : ℝ, ∀ R : ℝ, 1 < R → ∀ x, ‖fderiv ℝ (χ R) x‖ ≤ C

noncomputable def WSq (w : ℝ → ℝ) : EReal :=
  limsup (fun R : ℝ => ((w R / (4 * R ^ 2) : ℝ) : EReal)) atTop
```

```
def IsPeriodic (Λ : Set Plane) (E : Plane → Plane) : Prop :=
  ∀ v ∈ Λ, ∀ᵐ x ∂(volume : Measure Plane), E (x + v) = E x

noncomputable def triScale : ℝ := Real.sqrt (2 / Real.sqrt 3)

noncomputable def e₁ : Plane := !₂[1, 0]
noncomputable def e₂ : Plane := !₂[1 / 2, Real.sqrt 3 / 2]

def triLattice : Set Plane :=
  {x | ∃ m n : ℤ, x = triScale • ((m : ℝ) • e₁ + (n : ℝ) • e₂)}

def SandierSerfatyStatement : Prop :=
  (∃ χ, IsSquareCutoffFamily χ) ∧
    ∃ E₀ : Plane → Plane, Admissible triLattice E₀ ∧ IsPeriodic triLattice E₀ ∧
      ∀ χ : ℝ → Plane → ℝ, IsSquareCutoffFamily χ →
        ∃ w₀ : ℝ → ℝ, (∀ R : ℝ, 1 < R → HasLocalEnergy triLattice E₀ (χ R) (w₀ R)) ∧
          WSq w₀ ≠ ⊤ ∧ WSq w₀ ≠ ⊥ ∧
          ∀ (Λ : Set Plane) (E : Plane → Plane) (w : ℝ → ℝ), Admissible Λ E →
            (∀ R : ℝ, 1 < R → HasLocalEnergy Λ E (χ R) (w R)) → WSq w₀ ≤ WSq w

noncomputable def wTriClosedForm : ℝ :=
  Real.pi * Real.log (2 * Real.sqrt 2 * Real.pi / (Real.sqrt 3 * Real.Gamma (1 / 3) ^ 3))

def SandierSerfatyValueStatement : Prop :=
  (∃ χ, IsSquareCutoffFamily χ) ∧
    ∃ E₀ : Plane → Plane, Admissible triLattice E₀ ∧ IsPeriodic triLattice E₀ ∧
      ∀ χ : ℝ → Plane → ℝ, IsSquareCutoffFamily χ →
        (∃ w₀ : ℝ → ℝ, (∀ R : ℝ, 1 < R → HasLocalEnergy triLattice E₀ (χ R) (w₀ R)) ∧
            WSq w₀ = (wTriClosedForm : EReal)) ∧
          ∀ (Λ : Set Plane) (E : Plane → Plane) (w : ℝ → ℝ), Admissible Λ E →
            (∀ R : ℝ, 1 < R → HasLocalEnergy Λ E (χ R) (w R)) → (wTriClosedForm : EReal) ≤ WSq w

def WeakDivFree (j : Plane → Plane) : Prop :=
  ∀ φ : Plane → ℝ, IsTestFn φ → ∫ x, (j x 0 * pd 0 φ x + j x 1 * pd 1 φ x) = 0

def WeakCurlSS (m : ℝ) (Λ : Set Plane) (j : Plane → Plane) : Prop :=
  ∀ φ : Plane → ℝ, IsTestFn φ →
    ∫ x, (j x 0 * pd 1 φ x - j x 1 * pd 0 φ x) = 2 * Real.pi * (∑ᶠ p ∈ Λ, φ p) - m * ∫ x, φ x

def AdmissibleSS (m : ℝ) (Λ : Set Plane) (j : Plane → Plane) : Prop :=
  LocFinite Λ ∧ DensityBounded Λ ∧ LocallyIntegrable j volume ∧ WeakDivFree j ∧ WeakCurlSS m Λ j

def IsBallCutoffFamily (χ : ℝ → Plane → ℝ) : Prop :=
  (∀ R : ℝ, 1 < R →
      ContDiff ℝ ∞ (χ R) ∧ HasCompactSupport (χ R) ∧ (∀ x, 0 ≤ χ R x ∧ χ R x ≤ 1) ∧
        tsupport (χ R) ⊆ Metric.ball 0 R ∧ ∀ x ∈ Metric.closedBall (0 : Plane) (R - 1), χ R x = 1) ∧
    ∃ C : ℝ, ∀ R : ℝ, 1 < R → ∀ x, ‖fderiv ℝ (χ R) x‖ ≤ C

noncomputable def WBall (w : ℝ → ℝ) : EReal :=
  limsup (fun R : ℝ => ((w R / (Real.pi * R ^ 2) : ℝ) : EReal)) atTop

def ssTriLattice (m : ℝ) : Set Plane :=
  {x | ∃ a b : ℤ, x = Real.sqrt (4 * Real.pi / (m * Real.sqrt 3)) • ((a : ℝ) • e₁ + (b : ℝ) • e₂)}

def SandierSerfatyOriginalStatement : Prop :=
  (∃ χ, IsBallCutoffFamily χ) ∧
    ∀ m : ℝ, 0 < m →
      ∃ j₀ : Plane → Plane, AdmissibleSS m (ssTriLattice m) j₀ ∧ IsPeriodic (ssTriLattice m) j₀ ∧
        ∀ χ : ℝ → Plane → ℝ, IsBallCutoffFamily χ →
          ∃ w₀ : ℝ → ℝ, (∀ R : ℝ, 1 < R → HasLocalEnergy (ssTriLattice m) j₀ (χ R) (w₀ R)) ∧
            WBall w₀ ≠ ⊤ ∧ WBall w₀ ≠ ⊥ ∧
            ∀ (Λ : Set Plane) (j : Plane → Plane) (w : ℝ → ℝ), AdmissibleSS m Λ j →
              (∀ R : ℝ, 1 < R → HasLocalEnergy Λ j (χ R) (w R)) → WBall w₀ ≤ WBall w
```

Design choices [C, SSA-C-0001]. The plane is `EuclideanSpace ℝ (Fin 2)` (Euclidean norm; the product type $\mathbb{R} \times \mathbb{R}$ would carry the sup norm); test functions are `ContDiff` of order ∞, not $\top$ (which means analytic in current Mathlib). Λ is a `Set`, so points are simple; `LocFinite` is read for SS's "discrete". W is an `EReal`-valued `limsup`, so $\pm\infty$ are allowed as in the manuscript. The triangular minimizer is an *existential witness*: a $\Lambda_\triangle$-periodic admissible field with charges exactly on $\Lambda_\triangle$; SS2012 Proposition 3.1 [L, SSA-L-0003] makes this equivalent to using $E_\triangle$ (if the conjecture holds, $E_\triangle$ is a witness; conversely a witness is $E_\triangle + c$ and $W(E_\triangle) \le W(E_\triangle + c) \le W(E)$), and it avoids the periodic Green function Mathlib lacks. The quantifier order is: one E_0, then for every cutoff family a value w_0 with $W(E_0)$ finite and $\le W(E)$ for every competitor. Twelve definition-by-definition comparisons with the manuscript and SS2012 (`STATEMENT_FIDELITY.md`): rows 1–4, 6a, 7, 8 and 10 match; the three stated deviations are in row 5 (the existence of the puncture limit is assumed, conditional), row 6b (ball cutoffs restricted to smooth ones) and row 9 (the existential witness for $E_\triangle$); rows 11

and 12 (the value identification and the equivalence of the E- and j-forms) are not formalized. The independent audit re-compared the Lean definitions with the manuscript and with SS2012 and found no misstatement beyond these disclosed deviations; that is again a Tier C comparison by an agent.

Listing 2: The ten non-degeneracy lemmas (statements verbatim, proofs elided; proofs are in the file).

```
theorem negPi_admissibleBg_iff (b : ℝ) : AdmissibleBg b ∅ negPi ↔ b = 2 * Real.pi := -- proof elided
theorem negPi_admissible : Admissible ∅ negPi := -- proof elided
theorem negPi_not_admissible_noBackground : ¬ AdmissibleBg 0 ∅ negPi := -- proof elided
theorem negPi_not_admissible_wrongSign : ¬ AdmissibleBg (-(2 * Real.pi)) ∅ negPi := -- proof elided
theorem negPi_hasLocalEnergy {χ : Plane → ℝ} (hχ : Continuous χ) (hs : HasCompactSupport χ) :
    HasLocalEnergy ∅ negPi χ ((1 / 2) * ∫ x, χ x * ‖negPi x‖ ^ 2) := -- proof elided
theorem competitor_nonvacuous {χ : ℝ → Plane → ℝ} (hχ : IsSquareCutoffFamily χ) :
    ∃ (Λ : Set Plane) (E : Plane → Plane) (w : ℝ → ℝ), Admissible Λ E ∧
      ∀ R : ℝ, 1 < R → HasLocalEnergy Λ E (χ R) (w R) := -- proof elided
theorem triBasis_covolume : triScale ^ 2 * (e₁ 0 * e₂ 1 - e₁ 1 * e₂ 0) = 1 := -- proof elided
theorem ssBasis_covolume {m : ℝ} (hm : 0 < m) :
    Real.sqrt (4 * Real.pi / (m * Real.sqrt 3)) ^ 2 * (e₁ 0 * e₂ 1 - e₁ 1 * e₂ 0) = 2 * Real.pi / m := -- proof elided
theorem exists_squareCutoffFamily : ∃ χ, IsSquareCutoffFamily χ := -- proof elided
theorem exists_ballCutoffFamily : ∃ χ, IsBallCutoffFamily χ := -- proof elided
```

What is kernel-checked [A, SSA-A-0001..0006]. Listing 2 gives the ten lemmas. Their positive control is $E = -\pi x$ (class non-empty), their negative controls are the background-free and wrong-sign variants, and they use a bump function of positive integral (so the test class is not collapsed to 0). `competitor_nonvacuous` shows only that the competitor hypotheses (admissibility and the puncture limit for all $R > 1$) are *satisfiable*. Its witness is the charge-free field $-\pi x$, whose energy is $+\infty$ (by Gauss's law the flux of E through ∂K_r is $-8\pi r^2$, so $\int_{\partial K_r} |E|^2 \geq 8\pi^2 r^3$ and the local energy grows like R^4 [C], found by the audit), so the inequality $W(E_0) \leq W(E)$ holds for it trivially; nothing is shown about that inequality constraining any competitor. An earlier gloss in the Lean docstring and in the ledger said that the conclusion "constrains at least one competitor"; that was wrong and has been corrected (Lean docstring only: the theorem statements and the locked statement region are unchanged). Compilation of the statement module: rc 0 in 7.8 s (stage S3: rc 0 in 8.4 s); of the controls: rc 0 in 4.6 s (stage S3: rc 0 in 4.9 s) (the second figure in each pair is the stage-S3 run). The revision-stage rerun's axiom output is identical to the S3 transcript, the controls' output is identical, the file hashes match the locked values, and `statement_lock.py check` gives `match: true`. The locked statement-region hash is `38974cc16e90e77ac1553bedf6907ff2eb53f7689df00a9a82668d11574f0421`. Changing the background constant in `Admissible` from 2π to 0 changes the region hash (S3 lock control, region hash $\to$ `ed68e1c8...`).

Listing 3: Verbatim `#print axioms` output of the revision-stage rerun of the statement module; `PSS` abbreviates `ProgSandierSerfaty`.

```
'PSS.negPi_admissibleBg_iff' depends on axioms: [propext, Classical.choice, Quot.sound]
'PSS.negPi_admissible' depends on axioms: [propext, Classical.choice, Quot.sound]
'PSS.negPi_not_admissible_noBackground' depends on axioms: [propext, Classical.choice, Quot.sound]
'PSS.negPi_not_admissible_wrongSign' depends on axioms: [propext, Classical.choice, Quot.sound]
'PSS.negPi_hasLocalEnergy' depends on axioms: [propext, Classical.choice, Quot.sound]
'PSS.triBasis_covolume' depends on axioms: [propext, Classical.choice, Quot.sound]
'PSS.ssBasis_covolume' depends on axioms: [propext, Classical.choice, Quot.sound]
'PSS.exists_squareCutoffFamily' depends on axioms: [propext, Classical.choice, Quot.sound]
'PSS.exists_ballCutoffFamily' depends on axioms: [propext, Classical.choice, Quot.sound]
'PSS.competitor_nonvacuous' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Listing 4: The planted `sorry` and planted-axiom controls (file `SandierSerfatyStatementControls.lean`, docstrings elided) and, below it, the output of its compilation (the file compiles with rc 0; `#print axioms` catches both).

```
import Programmes.SandierSerfatyStatement

namespace ProgSandierSerfaty

theorem ctl_sorry : SandierSerfatyStatement := sorry

axiom ctl_cheat : False

theorem ctl_axiom : SandierSerfatyStatement := ctl_cheat.elim

end ProgSandierSerfaty

#print axioms ProgSandierSerfaty.ctl_sorry
#print axioms ProgSandierSerfaty.ctl_axiom
```

```
formal_cert/Programmes/SandierSerfatyStatementControls.lean:13:8: warning: declaration uses `sorry`
'PSS.ctl_sorry' depends on axioms: [propext, sorryAx, Classical.choice, Quot.sound]
'PSS.ctl_axiom' depends on axioms: [propext, Classical.choice, PSS.ctl_cheat, Quot.sound]
```

Honest gaps of the non-vacuity check. (1) No proof that `triLattice` is locally finite with bounded density. (2) No admissible field with charges is constructed, so the term $\sum_{p\in\Lambda}\varphi(p)$ in `WeakDivE` and the term $\pi\log\eta\sum\chi(p)$ in `localEnergyAt` are never exercised by a proof; the sign of the energy term is checked by hand only: near a charge $\frac{1}{2}\int_{\eta<|x-p|<\rho}|x-p|^{-2}=\pi\log\rho-\pi\log\eta$, so $+\pi\log\eta\,\chi(p)$ cancels the divergence, while $-\pi\log\eta$ or $2\pi\log\eta$ would make `HasLocalEnergy` unsatisfiable for every field with a charge in the support of χ [C]. (3) No $\Lambda_\triangle$-periodic admissible field is constructed (needs the torus Green function). Missing in Mathlib at the pinned revision: periodic Green function on a flat torus, Weyl's lemma for distributions, a packaged weak Gauss law for $x/|x|^2$.

4 Numerics: reproducing and attacking the finite certificate

4.1 Reproduction [X, exact integer; SSA-X-0001]

The manuscript's `verification/` folder contains `certificate.cpp`, `core.hpp`, `grids.hpp` and a wrapper `run-certificate.py` that compiles with `g++ -std=c++17 -O3 -fopenmp`, runs the program with `OMP_NUM_THREADS=32`, validates the order of the six stages and the three mesh markers, and writes a receipt. We ran the unchanged sources on cores 0–3 of a different host and toolchain (`g++ (Ubuntu 11.4.0-1ubuntu1 22.04.3) 11.4.0`, Python 3.10.12, Boost 1_74, Linux 6.8; the record: g++ 13.3.0, Boost 1_83, Python 3.12.13, Linux 5.15). The wrapper hard-codes 32 threads, so we oversubscribed four cores.

- Source hashes before and after equal the record (`certificate.cpp` `53d46bba38d671690b07b9893ea08feb5bdcea8039280c609be783f9e66a3bb2`, `core.hpp` `c7f76596cde648ae35707a242406a6b02d6b41bad6b19123aeca3aaec47dced3`, `grids.hpp` `4640e0a32eb20071800d408fc2b7b47269cdfcecfa99c862940c7afbbdaaa50b`, `run-certificate.py` `cbef4f4c082102a826e4b92c298c94fdfe3b5ff6622702b6226478142e1c7075`); sources unchanged: true.
- Wrapper exit 0, terminal line `OK`, stage validation passed; start 2026-10-25 20:23:19 UTC, end 2026-10-25 20:31:30 UTC; wall time 490.35 s (child timer 478.001 s: init_core 220.1 s, calcLam 2.8, calcK 0.3, bdsup 23.6, global_grid 125.4, inside_hess 105.7). The record's 72.6 s was on a 64-core host; no speed comparison is meant.
- **22 of 22 printed pairs equal the saved rows** of `certificate-record.txt` (sha256 `90e4f4055ce7f589303ad246dac23845db75f397aecbad71533230e34a86ee01`); Table 1 shows the rows the assertions use. Executable sha256 `ca7b5cae54a79c684f08c696a8a182fd9e4d9567701f6975596a26ac2dcddf1f`; log sha256 `cf6cc0c6fdf8175af35933f9000dfe8cca7edc5714e4922c152b011acb5208f8`.

The result is a reproduction of an exact-integer computation on a different toolchain. It is not Tier A, it does not test the manuscript's analytic bounds (audit, Section 5), and runtime trust (compiler, Boost, OpenMP, hardware) is assumed.

4.2 Negative controls and dose-response [X, SSA-X-0002]

We copied `verification/` (the corpus is never edited) and changed one line of `grids.hpp` in each copy (the wrapper copies take `OMP_NUM_THREADS` from the environment, set to 2, and accept that header; this is the only wrapper change). The midpoint-rule remainder in `er0` is $2\,S_{\max}H^2\chi/24$; the allowance in the negative-part assertion is `er_drop`$= S_{\max}(H/4\cdot G_s + D_h/2\cdot G_d)$. Table 2 shows the outcomes. The failing assertion in both rejected runs is the one at `grids.hpp:336`, `(gmin-factor).u>0 && (drneg+er_drop+K).v<0`. *Attribution:* every pair printed before the edited line is identical between the positive run and S1-neg (and between the positive run and B); only `factor` and `gmin-factor` (S1-neg), resp. `er_drop` and the final left side (B), change. Variant A passing through the same edited wrapper at 2 threads is the positive control for the wrapper edits: they are not the cause of the aborts. Neither rejected run reached `inside_hess`, so no negative control exercises the Hessian stage.

Margins. Written down before A and B finished (`predictions_AB.txt`, 20:43:39 UTC): A passes with `factor`$_{hi}\approx$ 20–21 (observed 20); B fails with a final left side $\approx -4585+5343=+758$ (observed 758). The

Table 1: Printed pairs (floor/ceil of the exact endpoints, units 10^{-7}) in the manuscript's saved record and in our run [X].

quantity	record	our run	equal
λ	(−67300312, −67300275)	(−67300312, −67300275)	yes
μ	(4138830, 4138838)	(4138830, 4138838)	yes
K	(−34823, −34809)	(−34823, −34809)	yes
$B(\ell)$	(10464643, 10464644)	(10464643, 10464644)	yes
$f_0(\ell)$	(−12135966, −12135965)	(−12135966, −12135965)	yes
slope	(774490, 774509)	(774490, 774509)	yes
$f(\ell)$	(44755, 44772)	(44755, 44772)	yes
gmin	(134, 162)	(134, 162)	yes
drneg	(24879, 24882)	(24879, 24882)	yes
factor	(0, 15)	(0, 15)	yes
`er_drop`	(0, 5343)	(0, 5343)	yes
gmin−factor	(120, 162)	(120, 162)	yes
drneg+`er_drop`+K	(−9944, −4585)	(−9944, −4585)	yes
g_{dd}	(10529222, 18021395)	(10529222, 18021395)	yes
g_{xx}	(128335, 864224)	(128335, 864224)	yes
det Hess	(86252, 1342330)	(86252, 1342330)	yes

Table 2: Positive run and three loosened copies. Pairs in units 10^{-7}; "rejected" means the program's own `assert` aborted (wrapper exit −6, `terminal_OK` false, stage validation failed). Last column: wrapper wall time in seconds (2 threads for the copies).

run	factor	gmin−factor	`er_drop`	drneg+`er_drop`+K	outcome	s
positive run (unchanged source)	(0, 15)	(120, 162)	(0, 5343)	(−9944, −4585)	accepted, `OK`	490
S1-neg: midpoint constant $1/24 \to 1$ (×24)	(0, 142)	(−7, 162)	(0, 5343)	(−9944, −4585)	**rejected** (SIGABRT, exit −6)	683
A: midpoint constant $1/24 \to 1/12$ (×2)	(0, 20)	(114, 162)	(0, 5343)	(−9944, −4585)	accepted, `OK`	1009
B: `er_drop` allowance ×2	(0, 15)	(120, 162)	(0, 10685)	(−9944, 758)	**rejected** (SIGABRT, exit −6)	874

midpoint constant therefore survives a loosening of about ×22.6 (`er0` ≈ 5.52, `eri` ≈ 9.48 in these units), whereas the Lipschitz allowance `er_drop` (5343 against a slack of 4585) survives only about ×1.86. The audit therefore re-derived that step (Section 5) and found its two terms correct as stated. This is an observation about thinness, not a defect.

4.3 Independent re-computation of the analytic constants [X, SSA-X-0003, SSA-X-0004, SSA-X-0005]

Without the C++, (a) the exact identity (3) can be tested: the printed λ, μ, K give $6K + \lambda + 2\pi\mu \in [-4.15042142, -4.15040429]$ (exact Fractions, width 1.71265e-5), which contains $W(E_\triangle) = -4.1504128076789633794$ computed both from the BS2018 closed form and from an Ewald sum (they differ by 2.1382e-50). Positive controls: the SS-normalized Ewald values -0.20108945 (triangular) and -0.1957972 (square) match SS2012 Remark 1.5 (≈ -0.2011, -0.1958) [L], and C from BS Theorem 1.5 equals C_{BHS} to 4.0091e-51. Negative controls, all outside the interval: the square lattice (-4.11716061233), no 2π scaling (-0.201089447612), the $\frac{1}{4}\log 2\pi$ shift dropped (-1.26348226266). *This check is blind to λ and μ:* each of the six regular sectors has area $1/6$ ($= p\,x_0$) and angle $\pi/3$ (the manuscript's Lemma K7), so the contribution of a sector is $K + \lambda/6 + \mu\pi/3$, the sum over six sectors is $6K + \lambda + 2\pi\mu$, and the λ and μ terms of K cancel exactly. (b) We therefore re-implemented f_0, e, G, λ, μ, K from the TeX formulas only (mpmath, 45-digit Gauss–Legendre): $\lambda = -6.73002928856$, $\mu = 0.413883384579$, $K = -0.00348158666$, each inside the certificate's printed pair, with $|G(p, x_0)| = 1.008 \times 10^{-53}$ (a positive control: $G(p, x_0) = 0$ by the definition of μ, so a

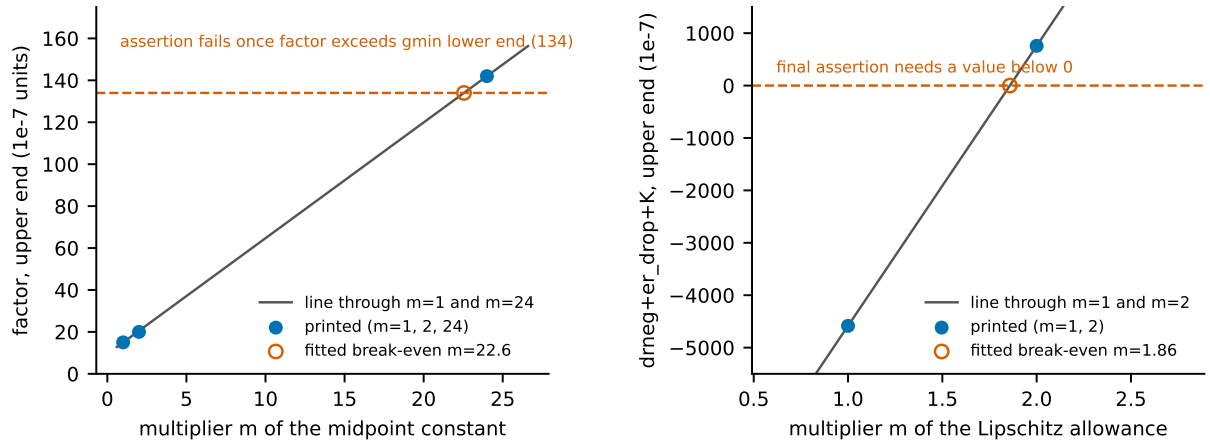


Figure 1: [X] Dose-response of the two thin margins. Left: upper end of `factor` against the multiplier of the midpoint constant; $m = 1, 2, 24$ are printed values, the line passes through $m = 1$ and $m = 24$ (so it predicts 20.5 at $m = 2$, observed 20) and crosses the lower end of gmin (134) at the fitted $m \approx 22.6$. Right: upper end of the final left side against the multiplier of the Lipschitz allowance: -4585 at $m = 1$, 758 at $m = 2$ (predicted 758), fitted break-even 1.86. The break-even points are *fitted* to coarsened printed pairs (the S1-neg numbers were known when the multipliers were estimated; the right panel has two points), not blind predictions.

value this small is the expected outcome) and identity residual $6K + \lambda + 2\pi\mu - W(E_\triangle) = -1.034 \times 10^{-25}$. Negative controls fall outside: without the angular term $\lambda = -6.752949$, $K = -0.0039749$; with mode $m = 1$ dropped $\lambda = -6.765096$, $K = 0.0005261$ (positive). Independently $B(\ell) = 1.04646435626$ (printed pair $(1.0464643, 1.0464644)$), $|c_1| = 0.634692 < 1$; dropping $m = 1$ gives 1.0827, a wrong lattice scale 1.0278. (c) Every hand constant of Section 5 passes in exact rational arithmetic (Step 8). Two errors in *our* scripts were caught and fixed: a mistyped compact form of the BS2018 closed form (corrected to the expanded form on p. 26), and a 25-digit tanh–sinh quadrature that cancelled catastrophically ($\lambda \approx 5.5 \cdot 10^5$); the failed run still satisfied the identity to $3 \cdot 10^{-20}$, which is how we know check (a) cannot see λ. This is evidence that the C++ evaluates the manuscript's formulas on $[p, R_0]$. It does not cover the spline pieces below p or above R_0, nor the grid assertions.

4.4 Improvement to LeanFlow: Lean data certificates, fail-closed for rendered sources [A on arithmetic only; X for the tests; SSA-A-0007, SSA-X-0006, SSA-X-0007]

Method. The new package `leanflow_mcp.lean` (LeanFlow commit `84e0c2899830fba671022eefcd40 8af9ecd565c8`, branch `feat/lean-data-certificates-2026-10-25`, local, not pushed; documentation `docs/LEAN_CERTIFY.md`) turns a claim — closed rational enclosures $x = [\text{lo}, \text{hi}]$ and linear inequalities over their endpoints — into one Lean theorem per assertion of the form `theorem cert_i : (0:Int) < D ∧ N * RD ≤ RN * D := by decide` (or `norm_num`), with all denominators cleared by Python from exact integer literals so that Lean does every multiplication and comparison. The checker compiles with a configured toolchain (timeout, optional memory cap), *appends* `#print axioms cert_i` itself, followed by a per-run random sentinel line, and requires exactly one report per theorem inside the allowlist and the sentinel exactly once after the last report; an exact `Fraction` oracle is a second evaluator and any disagreement is an error. *Fail-closed behaviour.* Each of the following returns `status≠ok`, with a test: missing toolchain, working directory or build target (including a real `import` of a nonexistent module), non-zero exit, kill by signal, exit 0 with no axiom report, timeout, memory cap, `sorry` warning, `sorryAx` without a warning, a custom axiom, `native_decide`, a forged second axiom report, `#exit` (alone, or together with a forged report; see the audit finding below), an invalid claim (floats, injected text, lo > hi; Lean never starts), an internal exception, and Lean/oracle disagreement. False statements give `rejected` with the failing theorem names. A finding recorded in the tests: on Lean 4.34.1 `native_decide` is reported as `t._native.native_decide.ax_1_1`, not `Lean.ofReduceBool`, so a classifier keyed to the old name would label it a generic custom axiom. *Results.* `pytest` gives 88 passed in 21.72 s on the revision-stage

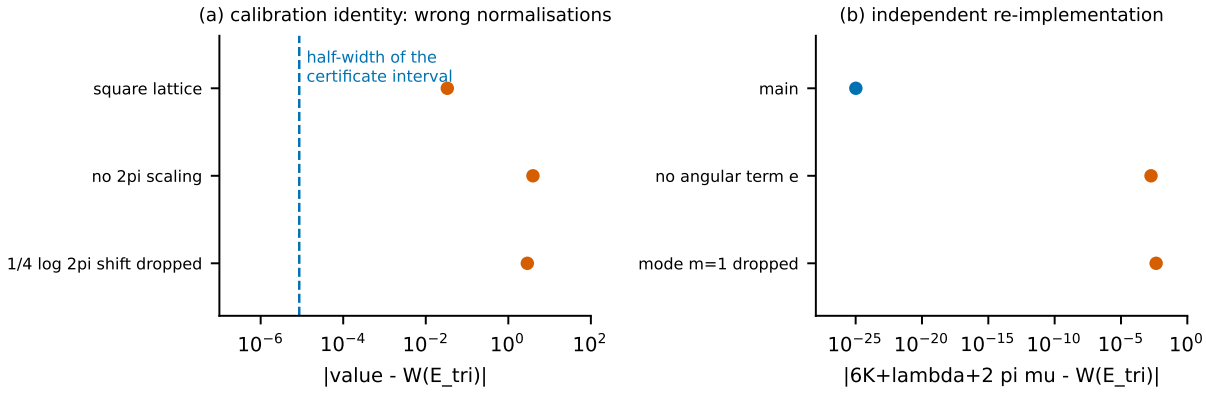


Figure 2: [X] (a) Wrong normalizations of the calibration identity miss $W(E_\triangle)$ by more than three orders of magnitude more than the half-width of the certificate interval (dashed). (b) Our independent re-implementation satisfies the identity to 10^{-25}; the two negative controls miss it by 10^{-3}.

rerun (84 pure-Python and core-Lean tests, including a 300-example Hypothesis property that the rendered `Int` statement equals the exact `Fraction` oracle, and 4 passed, 84 deselected pinned tests on Lean 4.34.1 with Mathlib). Applied to our runs: the 13 assertions about the printed pairs of the positive run are accepted (13 of 13, axiom lists empty, Lean 4.34.1); for the S1-neg run 7 of 10 are accepted and 3 are rejected, exactly those containing gmin−factor (`gmin - factor > 0 (m_* - E_0 - E_I > 0)`; `m_* - E_0 - E_I >= 1.20e-5`; `gmin.lo - factor.hi > 0 on printed pairs`). *Audit finding and fix (checker for arbitrary sources) [X, SSA-X-0007].* The independent audit (`AUDIT.md`, blocking item B2) probed the exported function `check_lean_source` at the pre-fix commit `542cea97aa78e30d8935ec81278aeb697a9f5dc7` with a source made of a custom `axiom cheat : False`, a theorem `(3:Int) < 2` proved from it, a forged `#eval IO.println` line imitating an axiom report, and `#exit`. `#exit` stops Lean before the appended `#print axioms`; Lean only warns "using 'exit' to interrupt Lean", which the checker did not reject, so the forged line was the single report and the result was `status ok, verified true` for a false statement. Controls in the same probe behaved as designed: the clean `decide` proof was accepted; the cheat without `#exit` gave `custom-axiom`; the cheat with the forged line but no `#exit` gave `axiom-report-ambiguous`; the cheat with `#exit` but no forgery gave `axiom-report-missing`. The generator path `certify()` was *not* affected (claim names are regex-checked, labels are single-line `-` comments; the audit's injection probe with labels `/-`, `-/ #exit` and `#exit` gave `rejected`), so the 13/13 and 3/10 results above stand, and were re-run unchanged with the fixed checker (the sentinel fix is commit `2c168be2de8104dddc48daecee1423c8d8f076b7`; commit `84e0c2899830fba671022eefcd408af9ecd565c8` adds only a docstring and documentation follow-up; same statuses, same accepted and rejected assertions). The fix appends a per-run random sentinel after the audit lines and requires it exactly once after the last axiom report, and also refuses the `#exit` warning (`exit-command`); there are 6 new tests (two with a fake toolchain, four with real Lean), and the audit's probe now returns `error / exit-command` for the forged source with the other four controls unchanged. The earlier statement in the documentation that "the source cannot choose what is audited" was too strong and has been corrected. *Residual scope:* Lean's `#eval` runs code, so this is a strong check but not a sandbox: a source written to defeat it could read back the temporary file and reproduce the sentinel, or redefine command syntax or elaborators (`macro_rules`, `elab_rules`) to forge the output; we therefore claim fail-closed behaviour only for sources rendered by the generator, and treat the checker as defence in depth for foreign sources. *Scope.* Tier A covers the arithmetic on the printed outward-rounded integers, not that the C++ computed them ([C, SSA-C-0005]); a failed `decide` is a rejection, not a kernel-checked negation of anything about the manuscript, so it is filed at Tier X. A toolchain that lies cannot be detected: the toolchain path is trusted configuration; real-Lean tests skip with a stated reason if the toolchain is absent; the pinned fixture needs about 78 s on a cold cache.

Table 3: Load-bearing citations: statements read on the stated page [L]; proofs are not re-checked.

used for	statement	source	read at
$\min_{\mathcal{A}_1} W$ achieved, finite; square-periodic minimizing sequence	Thm 1(3),(4)	SS2012 p. 5	paper stage (SSA-L-0001)
mass displacement, one g for every χ	Prop. 4.9 (4.12)	SS2012 p. 41	paper stage (SSA-L-0002)
periodic j minimized by j_Λ; scaling (1.9)	Prop. 3.1; Remark 7	SS2012 pp. 26, 3	paper stage (SSA-L-0003)
local L^r bound	Lemma 4.7	SS2012 pp. 38–39	audit stage only
$C = \frac{1}{\pi}\min W + \frac{\log\pi}{2} + \log 2$; value -4.1504128	Thm 1.5, Thm 7.7, (2.4)	BS2018 pp. 3, 4, 26	paper stage (SSA-L-0004)
$C_{\log,2}$	Conj. 4	BHS2012 p. 13	paper stage (SSA-L-0005)

5 Audit of the reduction and the certificate

Method and verdict [C, SSA-C-0002]. An AI agent read the manuscript sections (their sha256 values are in `REDUCTION_AUDIT.md`), re-derived each step, classified it CONFIRMED / GAP / CANNOT_ASSESS, and compared each certificate stage with the inequality and the constants it must certify. **Verdict: no step of Sections 2–5 was classified as a gap.** Counts: 10 steps for Section 2 (R1–R10), 7 for Section 3 (G1–G7), 8 for Section 4 (K1–K8), 6 for Section 5 (T1–T6), and 14 certificate-stage rows, all CONFIRMED. One context citation is CONFIRMED_WITH_MINOR_FIX: the manuscript's pin of Lieb–Rougerie–Yngvason "Lemma 3 and Appendix A" does not match the numbering of arXiv:1701.09064v3 (the content is eq. (3.53) and Proposition 4.2); it is context only. Three observations that are not defects: the strip estimate is $o(t^2/\log t)$, which is exactly what makes the $n\log n$ error $o(t^2)$ (Step 3); SS2012 Lemma 4.7's proof ends with $2W$ rather than W on p. 39, a factor the manuscript notes and its weakening covers; and the C++ midpoint error $H^3|A[2]|/12$ with $A[2] = Y''/2$ equals the manuscript's $(u-v)^3 \sup|Y''|/24$, so there is no factor-2 discrepancy.

The thinnest point. The negative-part assertion's Lipschitz allowance `er_drop` $\le 5343\cdot 10^{-7}$ is spent against a slack of $4585\cdot 10^{-7}$. The audit re-derived both terms: $\max(0,-y)$ is 1-Lipschitz and $|\partial_s G| \le L_2$, so the midpoint error per cell is $L_2 h^2/4$, giving $0.65\,S_h L_2/4$; moving to the nearest d-node costs at most $0.65\,L_1 D_h/2$. They are correct as stated, and the code matches. Our Variant B shows the margin is real: doubling the allowance breaks the certificate.

Not assessable (limits of the audit) [C, SSA-C-0004].

- *Code ↔ formula equivalence is not exhaustive.* The spline pieces on $[b,p)$ and $(R_0,l]$ (inner S-cutoff, P/Q extension, τ blending) and the grid-stage values (gmin, drneg, the L_1–L_4 hulls, the Hessian hulls) rest on code reading; jet `pow`/`log`/`sqrt`, `cpol` and `hor` were not read line by line. No independent recomputation covers them.
- *Runtime trust* (compiler, Boost, OpenMP, hardware). Our reproduction used a different toolchain and obtained the same integers, which is not a formal guarantee.
- *Internal proofs* of SS2012 Theorem 1, Lemma 4.7, Proposition 4.9 and BS2018 Theorem 1.5 are taken as cited.
- *Negative-control coverage:* no control exercised `inside_hess`, `calcLam` or `core_test`; a planted Hessian-breaking edit is still owed.
- *Check blindness:* the calibration identity is blind to λ and μ; this is why check (b) of Section 4 was added.
- *Uncited tool:* the strong minimum principle for $W^{2,2}_{\mathrm{loc}}$ supersolutions used in the separation lemma is standard but the manuscript gives no reference and we verified none. The independent audit located two published analogues, read on the cited pages of arXiv:1701.09064v3 (Lieb–Rougerie–Yngvason): the proof of their Proposition 4.2 (p. 21) uses the same superharmonic-minimum argument and refers to Lieb–Loss, *Analysis*, Chapter 9 (which nobody here checked), and their Lemma 4.4 (p. 22, minimal distance $\ge 1/\sqrt{\pi}$) is the closest published analogue of the separation lemma [L, audit reading only; not in the ledger]. The

manuscript's own pin ("Lemma 3 and Appendix A") could be replaced by eq. (3.53) (p. 20), Proposition 4.2 (p. 21) and Lemma 4.4 (p. 22), as read by the auditor.
- Section 7 (the corollary normalization) was checked at the level of the cited statements and the scaling re-derivation only.

6 Claim ledger

`ledger/build_ledger.py` files 25 claims: 7 at Tier A, 5 at L, 5 at C and 8 at X, each with a content-addressed evidence blob in `ledger/evidence/`: verbatim axiom lines and source hashes (A), verbatim excerpts in `ledger/sources/excerpts.json` (L), the argument text and its file hash (C), result files with `claim_licence: false` (X). Dependency edges respect the gate: A and L rows rest on nothing, the audit row (C) rests only on L rows and never on an X row, and the numerical rows rest on the scope rows. The Elenchus gate (`tools/ledger.py ledger.json --evidence-dir evidence`, output in `ledger/gate_output.txt`) exits with code 1. It reports **0 blocking findings** and **7 flags**, all `LEDGER_UNAUDITED_TIER_A`: no person has certified that the Tier A statements mean what their glosses say. The flags remain open. `elenchus_check.py` works on Lean files only and was not applicable; `citation_check.py` was run at the revision stage, on `pdftotext` output of the arXiv PDFs of SS2012 (v2) and BS2018 (v5) (`data/citation_check/`), for five theorem-type rows (SS Theorem 1 items 3–4, Theorem 2, Proposition 3.1, Proposition 4.9; BS Theorem 1.5): the verbatim passages are `ACCEPTED` (the quote is present in the primary text, statement and hypotheses listed; `statement_match` is our judgment), while the ledger's own excerpts are `REFUSED` because they are rejoined ASCII transcriptions of the alphaXiv extraction, not verbatim primary text, and two one-word tampers of real quotes are `REFUSED`. So the ledger's L rows are faithful transcriptions in content, not verbatim quotations; the tool does not check that a statement means what we say, and the conjectures (SS Conjecture 1, BHS Conjecture 4) and the BS value formula are outside its scope (they remain read-on-the-page claims by the auditor and the paper-stage agent).

7 Limitations

- **Conditionality.** Every statement about the truth of the conjecture rests on the openai/math manuscript (AI-generated, not peer reviewed), on cited literature whose proofs we did not re-check, and on a computer run whose runtime is trusted.
- **Same-pipeline verification.** Producer, auditor, Lean author and paper-stage agent are AI instances of the same pipeline; re-compilations are not producer≠verifier. No human has audited the Lean statements, the audit, or this paper.
- **The Lean statements are not proofs.** The ten lemmas only show that the definitions are not degenerate. The competitor class, the existence of the puncture limit and the triangular side's inhabitation are open points (Section 3); a subtle misstatement would not be caught by any Lean check.
- **Reproduction scope.** One run each, on cores 0–3 or 4–7 of one host; the wrapper's 32-thread setting oversubscribed four cores; no test of the analytic bounds the certificate depends on.
- **Control coverage and fitted margins.** The loosened copies change one constant each; the break-even multipliers are fitted to coarsened pairs and not blind; no control reaches `inside_hess`.
- **Independent audit and revision.** A second agent instance of the same pipeline audited this preprint before publication (verdict: accept with corrections; `AUDIT.md`); the corrections are listed in `REVISION.md`. Its open items remain open: no planted Hessian-breaking control, and no independent check of the spline pieces outside $[p, R_0]$ or of the grid-stage values.
- **LeanFlow checker scope.** Fail-closed behaviour is claimed only for sources rendered by the generator; the checker for arbitrary Lean sources is not a sandbox (Section 4.4).
- **Audit depth.** Section 5. In particular, `CONFIRMED` means the audit agent found no error, not that none exists.
- **Literature.** We did not search for work on the conjecture after the cited papers; claims about its status are the cited sources' own.

8 Reproducibility

AutoevolveAI files are uncommitted on top of `9d245742c1655852103abb28f585e21af54851e6` (branch `worktree-openai-math-discovery`). The openai/math checkout used for the corpus has HEAD `adc7f1241b42e322a6451854ab7e4b4c146bf78a`; the preprint is the folder `Triangular-minimality-for-planar-Coulomb-renormalized-energy-September-23-2026` (the four verification sources are identified by the sha256 values in Section 4). Lean sha256: statement `35add509990d50ebbdbbb70476fc652867d236a62534eda3d1021a414c21ae4d`; controls `aa94aca898a83529624931dd73dc7f0a3ab0ab31e78c9393de8c597661613895`. LeanFlow: commit `84e0c2899830fba671022eefcd408af9ecd565c8`.

```
tasket -c 4-7 python3 scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/SandierSerfatyStatement.lean --olean Programmes.SandierSerfatyStatement --json-out
       papers/prog_sandier_serfaty_audit/rerun/compile_statement.json
tasket -c 4-7 python3 scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/SandierSerfatyStatementControls.lean --json-out papers/prog_sandier_serfaty_audit/
       rerun/compile_controls.json
.venv/bin/python -I results/programmes/2026-10-25/P1/s3/statement_lock.py check
python3 -I -B run-certificate.py --output-dir <fresh dir>          # in the corpus verification/ folder, on a fresh directory; wrapper hard-codes OMP_NUM_THREADS=32
cd <LeanFlow worktree>; AutoevolveAI/.venv/bin/python -I -m pytest -c pyproject.toml tests -q -p no:cacheprovider
.venv/bin/python -I papers/prog_sandier_serfaty_audit/ledger/build_ledger.py; .venv/bin/python -I papers/prog_sandier_serfaty_audit/ledger/run_gate.py
.venv/bin/python -I papers/prog_sandier_serfaty_audit/gen_inputs.py; pdflatex main.tex (twice)
```

All data used by the paper (receipts, logs, patches, predictions, audit scripts and outputs, Lean transcripts) are copied into `papers/prog_sandier_serfaty_audit/data/`; the ledger evidence blobs hash them. *Paper-stage reruns* (same AI pipeline, not an independent verifier [X, SSA-X-0008]): Lean recompile of the statement (rc 0 in 7.8 s (stage S3: rc 0 in 8.4 s)) and controls (rc 0 in 4.6 s (stage S3: rc 0 in 4.9 s)), lock check `match: true`, LeanFlow 88 tests passed (after the checker fix; the pre-fix run, 82 tests at commit `542cea97aa78e30d8935ec81278aeb697a9f5dc7`, is kept as `rerun/leanflow_pytest_before_fix_commit542cea9.txt`). The revision changed the docstring of the Lean file (hash before: `d899416e3ed4189b10342e346a50454c629d4fffea057fe8a48ea19bf17476ff`; the locked statement-region hash is unchanged) and the pre-revision files are kept in `data/before_revision/`.

References

[1] E. Sandier, S. Serfaty, *From the Ginzburg–Landau model to vortex lattice problems*, Comm. Math. Phys. **313** (2012) 635–743; arXiv:1011.4617v2.

[2] L. Bétermin, E. Sandier, *Renormalized energy and asymptotic expansion of optimal logarithmic energy on the sphere*, arXiv:1404.4485v5 (2018).

[3] J. S. Brauchart, D. P. Hardin, E. B. Saff, *The next-order term for optimal Riesz and logarithmic energy asymptotics on the sphere*, Contemp. Math. **578** (2012) 31–61; arXiv:1202.4037v1 (the page numbers cited are those of the arXiv version).

[4] OpenAI, *Triangular minimality for planar Coulomb renormalized energy* (preprint dated 2026-09-23), in the OpenAI Math release, github.com/openai/math (checkout HEAD `adc7f124...`), folder `preprints/Triangular-minimality-for-planar-Coulomb-renormalized-energy-September-23-2026`. AI-generated; not peer reviewed.

[5] X. Callens, *The Triangular Lattice and Universal Optimality* (textbook), doi:10.5281/zenodo.23295142 (2026).

[6] X. Callens, five sibling preprints applying the same approach (kernel-checked consequences of the same openai/math release for explicit interactions, with Elenchus-tiered claims): bilayer dipolar excitons doi:10.5281/zenodo.23646103, Yukawa crystals doi:10.5281/zenodo.23646105, Rytova–Keldysh doi:10.5281/zenodo.23646107, $\sigma = 3$ dyadic enstrophy doi:10.5281/zenodo.23646111, Bogoliubov structure factor doi:10.5281/zenodo.23646117 (DOIs reserved on the title pages at the time of writing).

This preprint applies the tiered approach of [5, 6] to a claim about a different problem: here the openai/math result is the object of audit rather than an input.